БОУ г. Омска “ Лицей №74”

Индивидуальный проект на тему “обучающая программа в стиле 80 дней вокруг света на разном транспорте”

Выполнил: Сербин Сергей Александрович 9д класса
Руководитель: Сотникова Мария Владиславовна

Оглавление

1. Введение 3
2. Описание проекта 4
2.1. Жанр и целевая аудитория 4
2.2. Цели и задачи проекта 4
2.3. Аналоги (сравнение с существующими решениями) 5
2.4. Гипотеза 6
2.5. Методы, использованные в проекте 6
2.6. Практическая значимость 7
2.7. Цель игры (с точки зрения игрока) 8
2.8. Задачи проекта (с точки зрения разработки) 8
2.9. Технологический стек 9
3. Игровая механика 10
3.1. Игровой мир и система камеры 10
3.2. Управление высотой 10
3.3. Система ветровых зон (невидимых для игрока) 11
3.4. Визуальная индикация ветра 11
3.5. Таймер и полоса прогресса 12
4. Обучающая система 13
4.1. Слайдер-обучение 13
4.2. Система подсказок 14
4.3. Мини-игра «Сопоставь детали» 14
5. Препятствия и физика столкновений 15
6. Точки развития проекта 16
7. Реализация 17
8. Вывод 18
9. Список использованных источников 19
10. Приложение. Листинг программы 19

1. Введение

В современном мире информационных технологий особую актуальность приобретают обучающие компьютерные игры. Они позволяют совместить развлечение с получением новых знаний, что значительно повышает эффективность усвоения материала, особенно у детей и подростков. В отличие от традиционных учебников, игровая форма подачи информации вызывает больший интерес и мотивирует пользователя на активное взаимодействие с контентом.

Темой данного проекта является создание познавательной игры «80 дней вокруг света на разном транспорте», разработанной на языке программирования Python с использованием библиотеки Pygame. Игра знакомит пользователя с принципами воздухоплавания, физическими законами, лежащими в основе полёта воздушного шара (сила Архимеда, плотность воздуха, выталкивающая сила), а также с историей воздухоплавания.

На текущем этапе разработки реализован первый вид транспорта — воздушный шар. Игроку предстоит управлять шаром, перемещаясь по трём высотным зонам, ловить попутные порывы ветра, избегать столкновений с птицами и уложиться в отведённое время. Важная особенность игры: зоны с попутным ветром никак не подсвечиваются на экране. Игрок должен самостоятельно искать их, ориентируясь на визуальные эффекты (летящие белые полосы) и на изменение скорости движения шара.

Актуальность работы обусловлена тем, что качественных отечественных обучающих игр по физике для школьников крайне мало. Данный проект призван заполнить эту нишу, предлагая простой, интуитивно понятный и при этом научно достоверный игровой опыт.

2. Описание проекта

2.1. Жанр и целевая аудитория

Игра «80 дней вокруг света на на разном транспорте» выполнена в жанре обучающего симулятора с элементами аркады. Основной упор сделан на образовательную составляющую, однако присутствуют динамические элементы (таймер, препятствия), которые создают необходимый уровень напряжения и интереса.

Целевая аудитория:

- Школьники в возрасте от 10 до 14 лет
- Все интересующиеся историей воздухоплавания
- Пользователи, предпочитающие обучающие программы в игровой форме

Платформы: игра работает на Windows, Linux и macOS.

2.2. Цели и задачи проекта

Цель проекта:

Разработать познавательную компьютерную игру на языке Python, которая в интерактивной форме знакомит пользователя с физическими принципами полёта воздушного шара, историей воздухоплавания и устройством летательного аппарата.

Задачи проекта:

1. Изучить физические законы, лежащие в основе воздухоплавания (сила Архимеда, плотность воздуха, выталкивающая сила).
2. Изучить библиотеку Pygame и её возможности для создания 2D-игр.
3. Спроектировать архитектуру игры: игровой мир, камеру, систему управления.
4. Реализовать физическую модель движения шара с учётом трёх высотных зон.
5. Разработать систему невидимых ветровых зон с косвенной визуализацией (белые полосы).
6. Создать обучающий блок из 16 слайдов с автоматическим переносом текста.
7. Реализовать мини-игру «Сопоставь детали» для закрепления материала.
8. Добавить препятствия (птицы) и систему столкновений.
9. Внедрить таймер и полосу прогресса.
10. Написать код, соответствующий стандартам PEP 8, и подготовить документацию.

2.3. Аналоги (сравнение с существующими решениями)

На рынке образовательного программного обеспечения существуют игры и симуляторы, посвящённые воздухоплаванию и физике полёта. Однако большинство из них имеют ряд недостатков.

Основные аналоги:

Название	Платформа	Недостатки
Microsoft Flight Simulator	PC	Ориентирован на профессиональную аудиторию, сложное управление, требует мощного оборудования, не предназначен для обучения школьников
Kerbal Space Program	PC	Сложная физика орбитальной механики, высокий порог входа, не фокусируется на воздухоплавании
Simple Planes	PC, Mobile	Упор на конструирование, а не на обучение физике, интерфейс на английском языке
Воздушный шар (Flash-игры)	Веб	Устаревшие технологии, отсутствие образовательной составляющей, не работают без Flash

Отличие данного проекта:

- В отличие от перечисленных аналогов, разработанная игра:
- Ориентирована *исключительно на обучение* (физика, история, устройство шара)
- Имеет *понятный интерфейс* и пошаговое обучение
- Не требует мощного оборудования (работает на любом ПК с Python)
- Содержит *мини-игру для закрепления материала*
- Имеет *открытый исходный код*, что позволяет использовать его в образовательных целях

Таким образом, проект занимает нишу *доступного обучающего инструмента* для школьников, в отличие от существующих решений, которые либо слишком сложны, либо не имеют образовательной направленности.

2.4. Гипотеза

Гипотеза проекта:

Если представить физические законы и принципы воздухоплавания в виде интерактивной игры с элементами обучения, то пользователь (школьник) усвоит материал быстрее и эффективнее, чем при изучении традиционных учебников или лекций.

Обоснование гипотезы:

- Игровая форма подачи материала повышает мотивацию и вовлечённость.
- Интерактивное управление (кнопки балласта и клапана) позволяет на практике понять, как изменение массы и подъёмной силы влияет на высоту полёта.
- Система невидимых ветровых зон заставляет пользователя экспериментировать и искать закономерности, что способствует развитию исследовательских навыков.
- Встроенное обучение и мини-игра обеспечивают многократное повторение ключевой информации.

2.5. Методы, использованные в проекте

При разработке проекта были использованы следующие методы:

Метод	Описание
Анализ предметной области	Изучение физики воздухоплавания, истории братьев Монгольфье, существующих аналогов
Объектно-ориентированное программирование	Разработка классов MatchingGame, HintSystem, Bird для структурирования кода
Метод математического моделирования	Реализация физики движения шара, плавной интерполяции камеры, синусоидальной анимации
Метод итерационной разработки	Постепенное добавление функций: от базового движения шара до полной обучающей системы

Метод	Описание
Эксперимент (тестирование)	Проведение пробных запусков игры, выявление и исправление ошибок (багов)
Метод визуализации	Создание градиентного фона, анимированных птиц, эффекта ветра с помощью библиотеки Pygame
Декомпозиция	Разделение проекта на модули: игровой мир, управление, обучение, препятствия, визуальные эффекты

2.6. Практическая значимость

Разработанный проект имеет следующую практическую значимость:

1. Образовательная:

- Может использоваться на уроках физики в 7–8 классах при изучении темы «Сила Архимеда. Плавание тел».
- Содержит наглядную демонстрацию принципа работы воздушного шара.
- Включает исторические факты, расширяющие кругозор.

2. Методическая:

- Может служить примером для кружков программирования (создание 2D-игр на Python).
- Открытый исходный код позволяет студентам и школьникам изучать структуру проекта, принципы ООП, работу с графикой.

3. Социальная:

- Делает сложные физические понятия доступными для широкой аудитории.
- Способствует популяризации программирования среди школьников.
- Может распространяться бесплатно в образовательных учреждениях.

4. Техническая:

- Демонстрирует возможности библиотеки Pygame для создания полноценных игр с обучающей составляющей.
- Показывает реализацию системы невидимой механики (ветровые зоны) с косвенной визуализацией.

2.7. Цель игры (с точки зрения игрока)

Перед игроком ставится несколько последовательных задач:

1. Основная цель: долететь на воздушном шаре до финишной точки (города), преодолев расстояние в 3000 условных единиц.
2. Временное ограничение: уложиться в 60 секунд.
3. Условие безопасности: избегать столкновений с птицами.
4. Тактическая задача: находиться в зоне попутного ветра, чтобы двигаться вперёд быстрее. Поскольку зона не подсвечена, игрок должен искать её по белым летящим полосам.
5. Финальный этап: после достижения города необходимо безопасно приземлить шар.

2.8. Задачи проекта (с точки зрения разработки)

№	Задача	Реализация в коде
1	Создать масштабируемый игровой мир с прокруткой	WORLD_WIDTH = 5000, WORLD_HEIGHT = 4000, система камеры
2	Реализовать плавное управление высотой	set_target_zone(), update_balloon_position()
3	Создать систему невидимых ветровых зон	Переменная in_green_zone, случайная смена color_config
4	Добавить визуальную индикацию ветра (белые полосы)	pygame.draw.line() с динамической анимацией
5	Создать пошаговое обучение	Список slides из 16 элементов, wrap_text()
6	Реализовать мини-игру на запоминание деталей	Класс MatchingGame
7	Добавить препятствия (птицы) с анимацией	Класс Bird, проверка коллизий
8	Внедрить ограничение по времени	game_timer, круговой индикатор
9	Визуализировать прогресс	Полоса прогресса
10	Создать систему подсказок	Класс HintSystem

2.9. Технологический стек

Компонент	**Технология**	**Назначение**
Язык	Python 3.10+	Основная разработка
Графика	Pygame 2.0+	Отрисовка, ввод, спрайты
GIF-анимация	Pillow (PIL)	Извлечение кадров из GIF
Математика	math (встроенная)	Синусоидальная анимация
Случайные числа	random (встроенная)	Спавн облаков, птиц, смена зон
Частота кадров	60 FPS	Стабильная работа

3. Игровая механика

3.1. Игровой мир и система камеры

Игровой мир имеет размер 5000×4000 пикселей. Это позволяет реализовать ощущение длительного путешествия.

Камера реализована через смещение отрисовки всех объектов. Когда шар находится на высоте менее 300 пикселей от земли, камера привязана к земле. При подъёме выше этого порога камера начинает плавно следовать за шаром.

Код системы камеры:

```
CAMERA_TRIGGER_HEIGHT = 300

if current_height >= CAMERA_TRIGGER_HEIGHT:

    camera_active = True

    target_offset_x = balloon_x - WIDTH//2 + balloon_width//2

world_offset_x += (target_offset_x - world_offset_x) * camera_follow_speed
```

Элементы мира:

- 15 облаков, движущихся с разной скоростью
- Птицы с анимацией
- Гора и деревья
-

3.2. Управление высотой

Игрок изменяет высоту двумя кнопками в левом нижнем углу:

- Балласт — подъём вверх (сброс песка, уменьшение массы)
- Клапан — спуск вниз (выпуск горячего воздуха)

Всего три высотные зоны: верхняя (0), средняя (1), нижняя (2). Переключение плавное:

```
def update_balloon_position():

    target_y = get_zone_center_y(target_zone) - balloon_height // 2

    if abs(balloon_y - target_y) > move_speed:

        if balloon_y < target_y:

            balloon_y = min(balloon_y + move_speed, target_y)

        else:

            balloon_y = max(balloon_y - move_speed, target_y)
```

3.3. Система ветровых зон (невидимых для игрока)

Ключевая особенность: зона с попутным ветром никак не подсвечивается на экране. В коде она называется color_config, но это внутренняя переменная, не отображаемая пользователю.

В каждый момент времени только одна из трёх высотных зон даёт ускорение. Игрок не видит, какая это зона. Он может только:

1. Наблюдать за белыми полосами ветра
2. Следить за скоростью шара

Алгоритм работы:

1. На старте ветровая зона — средняя (индекс 1)
2. Шар получает ускорение wind_speed = 2.5, если current_zone == color_config
3. Каждые 500 метров пути ветровая зона случайным образом меняется

```
in_wind_zone = (current_zone == color_config) # невидимо для игрока

if in_wind_zone:

    balloon_x += wind_speed

    total_progress += wind_speed

    if int(total_progress / distance_for_color_change) > color_change_counter:

        change_colors() # случайный выбор новой зоны
```

3.4. Визуальная индикация ветра

Поскольку игрок не видит активную зону, разработчик добавил косвенную визуальную индикацию — летящие белые полосы. Они появляются только на высоте активной ветровой зоны.

```
time_offset = pygame.time.get_ticks() * 0.15

x_pos = (time_offset + i * 55 + row * 25) % (WIDTH + 300) - 150

pygame.draw.line(screen, (255, 255, 255), (line_x, y_pos), (line_x + length, y_pos), 2)
```

Что видит игрок:

- Белые полосы летят на определённой высоте — на этой высоте сейчас попутный ветер
- Шар ускорился — игрок попал в ветровую зону
- Полосы исчезли — игрок вышел из ветровой зоны

Такое решение сделано намеренно: игрок должен экспериментировать с высотой, а не просто следить за готовой подсказкой.

3.5. Таймер и полоса прогресса

Игра ограничена 60 секундами. Таймер отображается в виде кругового индикатора в левом верхнем углу.

Полоса прогресса находится в правом верхнем углу и показывает, какая часть пути пройдена:

```
fill_width = int((total_progress / TARGET_DISTANCE) * bar_width)

pygame.draw.rect(screen, (100, 255, 100), (bar_x, bar_y, fill_width, bar_height))
```

4. Обучающая система

4.1. Слайдер-обучение

Перед началом игры на экране появляется панель обучения справа. 16 слайдов:

0 — Приветствие

1 — Что такое воздушный шар

2 — Из чего состоит шар

3 — История воздухоплавания

4 — Как шар поднимается

5 — Сила Архимеда

6 — Почему не улетает в космос

7 — Управление: вверх и вниз

8 — Управление: ветер (зоны не видны, ищи по белым полосам)

9 — Мини-игра

10 — Твоя очередь

11-12 — Поздравление, Посадка

13-15 — Интересные факты

Техническая реализация:

- Данные в списке словарей
- Автоматический перенос текста (wrap_text())
- Тень у заголовка
- Пульсация цвета через math.sin()

```
def wrap_text(text, font, max_width):
    words = text.split(' ')
    lines = []
    current_line = ""
    for word in words:
        test_line = current_line + word + " "
        if font.size(test_line)[0] <= max_width:
            current_line = test_line
        else:
            lines.append(current_line)
            current_line = word + " "
    if current_line:
        lines.append(current_line)
    return lines
```

4.2. Система подсказок

При первом использовании функций появляются всплывающие подсказки. Каждая показывается один раз.

```
class HintSystem:
    def init(self):
        self.hints = {
            "first_wind_zone": {"text": "Ты в зоне ветра! Ищи белые полосы", "shown": False},
            "balast": {"text": "Сброс балласта = подъём вверх", "shown": False},
            "klapan": {"text": "Выпуск воздуха = спуск вниз", "shown": False},
            "bird": {"text": "Осторожно! Облетай птиц", "shown": False}
        }
```

4.3. Мини-игра «Сопоставь детали»

На слайде №9 запускается мини-игра. Игрок сопоставляет три детали с описаниями:

- Клапан — Выпускает горячий воздух
- Веревка — Соединяет корзину с оболочкой
- Горелка — Нагревает воздух

```
def check_match(self, item_idx, desc_idx):
    if item_idx == desc_idx:
        self.items[item_idx]["matched"] = True
        self.descriptions[desc_idx]["matched"] = True
        if all(item["matched"] for item in self.items):
            self.completed = True
    else:
        self.message = "Неправильно!"
```

5. Препятствия и физика столкновений

Птицы — движущееся препятствие. При столкновении — GAME OVER.

Характеристики:

- Анимированные (GIF или процедурная анимация)
- Летят навстречу шару
- Спавн: не чаще одной птицы на зону, максимум 3 птицы

```
bird_spawn_timer += dt

if bird_spawn_timer >= bird_spawn_delay:

    occupied_zones = [bird.zone for bird in birds]

    free_zones = [z for z in range(3) if z not in occupied_zones]

    if free_zones and len(birds) < max_birds:

        birds.append(Bird(zone_choice, zone_center_y, bird_frames))
```

Проверка столкновения: хитбокс шара уменьшен на 20% для честности.

```
hitbox_reduction = 0.8

balloon_rect = pygame.Rect(balloon_x - hitbox_width//2, balloon_y - hitbox_height//2,
hitbox_width, hitbox_height)

for bird in birds[:]:

    if bird.check_collision(balloon_rect):

        game_over = True
```

6. Точки развития проекта

Направление	**Что планируется**
Графика	Качественные спрайты, анимация взлёта/посадки, погодные эффекты
Контент	Дирижабль, пароход, поезд и другие разнообразные виды транспорта
Механики	случайные события, таблица рекордов, достижения
Звук	Музыка, звуки ветра, горелки, птиц
Техника	Оптимизация, сохранение прогресса, выбор сложности, локализация

7. Реализация

В ходе проекта создана полноценная познавательная игра на Python с использованием Pygame.

Реализовано:

- Физика движения шара с плавным переключением высоты
- Система невидимых ветровых зон с косвенной визуализацией (белые полосы)
- Управление через кнопки балласта и клапана
- Таймер 60 секунд с круговым индикатором
- Препятствия (птицы) с проверкой столкновений
- Обучение из 16 слайдов с автопереносом текста
- Мини-игра на сопоставление деталей
- Система контекстных подсказок
- Прокручиваемый мир с плавной камерой
- Визуальные эффекты: градиент, облака, анимированный ветер

8. Вывод

В ходе выполнения данного проекта была разработана полноценная познавательная игра «80 дней вокруг света на воздушном шаре» на языке Python с использованием библиотеки Pygame.

В процессе работы были решены следующие задачи:

1. Изучены физические законы, лежащие в основе воздухоплавания (сила Архимеда, плотность воздуха, выталкивающая сила), и реализована их программная модель.
2. Освоена библиотека Pygame, её возможности по отрисовке графики, обработке ввода, работе со спрайтами и анимацией.
3. Спроектирована архитектура игры, включающая игровой мир размером 5000×4000 пикселей, плавную систему камеры, три высотные зоны и две кнопки управления.
4. Реализована физическая модель движения шара с плавным переключением между высотными зонами.
5. Создана уникальная система невидимых ветровых зон: игрок не видит, где дует попутный ветер, и вынужден искать его по белым летящим полосам и ускорению шара. Это решение стимулирует исследовательское поведение и экспериментирование.
6. Разработано пошаговое обучение из 16 слайдов с автоматическим переносом текста, тенью у заголовков и пульсацией цвета.
7. Создана мини-игра «Сопоставь детали», закрепляющая знание устройства воздушного шара.
8. Добавлены препятствия — анимированные птицы, реализована система спавна и проверка столкновений с уменьшенным хитбоксом шара.
9. Внедрены таймер на 60 секунд с круговым индикатором и полоса прогресса.
10. Код написан в соответствии со стандартами PEP 8, снабжён комментариями и может использоваться в образовательных целях.

Подтверждение гипотезы:

Гипотеза проекта подтвердилась. Игровая форма подачи материала, интерактивное управление и система невидимых ветровых зон, требующая от игрока самостоятельного поиска закономерностей, делают процесс обучения более эффективным по сравнению с традиционными методами. Пользователь не просто читает теорию, а на практике осваивает принципы управления высотой и навигации воздушного шара.

9. Список использованных источников

1. Саммерфилд М. «Python на практике». — М.: ДМК Пресс, 2017.
2. Макграт М. «Программирование на Python для начинающих». — М.: Эксмо, 2019.
3. Документация Pygame: https://www.pygame.org/docs/
4. Документация Pillow: https://pillow.readthedocs.io/
5. Жюль Верн. «Вокруг света за 80 дней». — М.: Детская литература, 2018.

---

10. Приложение. Листинг программы

Полный код программы прилагается отдельно. Более 800 строк. Основные классы: MatchingGame, HintSystem, Bird. Основные функции: wrap_text(), update_balloon_position(), change_colors().